



**Tertiary Infotech Academy Pte Ltd**

UEN: 201200696W

# **LEARNER GUIDE**

For

## **Generative AI for Agile Project Management (C324)**

Course Code: C324

Conducted by

**Tertiary Infotech Academy Pte Ltd**

UEN: 201200696W

**Version 1.0**

## DOCUMENT VERSION CONTROL RECORD

| Version Number | Effective Date of Release | Summary of Included Changes                                                   | Author         |
|----------------|---------------------------|-------------------------------------------------------------------------------|----------------|
| 1.0            | 27 July 2026              | Initial release — C324 Generative AI for Agile Project Management courseware. | Dr. Alfred Ang |

## TABLE OF CONTENTS

|                                                                                |           |
|--------------------------------------------------------------------------------|-----------|
| <b>Introduction.....</b>                                                       | <b>4</b>  |
| <b>Course Learning Outcomes.....</b>                                           | <b>4</b>  |
| <b>Before You Start — Preparation.....</b>                                     | <b>5</b>  |
| <b>Topic 01 — Getting Started with Generative AI for Agile (25%).....</b>      | <b>6</b>  |
| Lab 1 — Set Up Your AI Toolkit for Agile Project Management.....               | 6         |
| Lab 2 — Write Effective Prompts and Build an Agile Prompt Library.....         | 8         |
| Lab 3 — Apply Responsible, Human-in-the-Loop AI Practices.....                 | 10        |
| <b>Topic 02 — AI for Backlogs, User Stories and Planning (25%).....</b>        | <b>12</b> |
| Lab 4 — Draft and Refine User Stories and Acceptance Criteria.....             | 12        |
| Lab 5 — Groom and Prioritise the Product Backlog with AI.....                  | 14        |
| Lab 6 — Plan the Sprint, Estimate, and Build the Release Roadmap.....          | 16        |
| <b>Topic 03 — AI for Sprints, Standups and Delivery (25%).....</b>             | <b>18</b> |
| Lab 7 — Summarise Stand-ups and Track Sprint Progress.....                     | 19        |
| Lab 8 — Identify Risks, Blockers and Dependencies with AI.....                 | 21        |
| Lab 9 — Assist Documentation, Communication and Testing.....                   | 22        |
| <b>Topic 04 — AI for Reporting, Retrospectives and Improvement (25%).....</b>  | <b>24</b> |
| Lab 10 — Generate Status Reports and a Delivery Dashboard.....                 | 25        |
| Lab 11 — Analyse Velocity and Metrics with AI.....                             | 27        |
| Lab 12 — Run an AI-Assisted Retrospective and Plan Continuous Improvement..... | 29        |
| <b>Wrap-Up.....</b>                                                            | <b>31</b> |
| <b>Next Steps.....</b>                                                         | <b>32</b> |
| <b>Glossary.....</b>                                                           | <b>32</b> |

## Introduction

This Learner Guide accompanies the Generative AI for Agile Project Management (C324) course, conducted by Tertiary Infotech Academy Pte Ltd. It carries the full detail of all 12 hands-on labs, in the order you will run them across the two days, together with the concepts each lab depends on.

The labs build a single, connected deliverable — the Tempo 2.0 Agile AI Playbook, for a fictional habit-tracking app called Tempo from the studio Cadence Labs, planning and delivering its 2.0 release. You start in Lab 1 by setting up ChatGPT, Claude and Gemini as an agile toolkit, then in every lab you take the delivery one stage further — a reusable prompt library, a responsible-AI checklist, refined user stories and acceptance criteria, a groomed and prioritised backlog, a sprint plan and release roadmap, stand-up summaries and progress tracking, a risk and dependency log, AI-assisted documentation and test scenarios, stakeholder status reports and a dashboard, a velocity analysis, and finally an AI-assisted retrospective and continuous-improvement plan. A Tempo 2.0 project brief with sample inputs is supplied in labs/reference-pack/; you may substitute your own non-confidential project wherever you prefer.

## Course Learning Outcomes

- LO1: Explain how generative AI assistants (ChatGPT, Claude, Gemini) support Agile and Scrum, and set up an AI toolkit and workspace for agile project management.
- LO2: Write effective, structured prompts for agile tasks and build a reusable prompt library the whole team can use.
- LO3: Apply responsible, human-in-the-loop AI practices — reviewing, fact-checking and protecting sensitive data — across agile work.
- LO4: Draft and refine clear, INVEST-quality user stories and acceptance criteria with AI assistance.
- LO5: Groom and prioritise a product backlog with AI using INVEST, MoSCoW and value-versus-effort thinking.
- LO6: Plan a sprint, estimate work, and generate a release roadmap and plan with AI support.
- LO7: Summarise stand-ups and track sprint progress with AI while keeping the team's judgement in the loop.
- LO8: Identify risks, blockers and dependencies with AI and plan mitigations and escalations.
- LO9: Assist documentation, communication, quality and testing across the sprint with AI.
- LO10: Generate status reports and a delivery dashboard for different stakeholders with AI.
- LO11: Analyse velocity and delivery metrics with AI to surface trends and honest insights.

- LO12: Run an AI-assisted retrospective and turn its output into a concrete continuous-improvement plan.

## Before You Start — Preparation

### What you need

- A laptop (Windows or Mac) with a modern web browser (Chrome, Edge, Safari or Firefox) and a reliable internet connection — every generative feature runs in the cloud.
- Access to at least one, ideally all three, generative AI assistants: ChatGPT (chat.openai.com), Claude (claude.ai) and Gemini (gemini.google.com). A free account for each is enough to follow the labs; the trainer will confirm what is available.
- A signed-in account for each assistant you will use, tested before Lab 1 with a simple 'hello' prompt so you know it responds.
- A place to keep your work — a documents folder or note-taking app — to save each reviewed AI output and prompt as a section of your Tempo 2.0 Agile AI Playbook.
- The supplied Tempo 2.0 project brief and sample inputs (product vision, team, raw backlog, sample stand-up notes and sprint metrics) in labs/reference-pack/ — or a few notes and sample inputs from your own non-confidential project to use instead.

### Verify your setup

Before Lab 1, confirm you can open and sign in to at least one assistant, send a simple prompt and get a reply, and that you have the Tempo 2.0 project brief to hand. If anything is missing, tell the trainer.

Open [chat.openai.com](https://chat.openai.com) (ChatGPT) · [claude.ai](https://claude.ai) (Claude) · [gemini.google.com](https://gemini.google.com) (Gemini)  
 · sign in · send "Hello, are you ready to help me with agile project management?" · confirm a reply

### Conventions used in every lab

- Placeholders such as <YOUR PROJECT>, <TEAM CAPACITY> or <PASTE NOTES> are replaced with your own values before you send a prompt.
- Prompts to paste into ChatGPT, Claude or Gemini are shown in the 'Prompt to use' blocks — adapt the bracketed parts to your own project.
- Where a lab says 'any assistant', use whichever of the three you prefer; a few labs ask you to compare the same prompt across two tools.
- Every lab ends with a 'Test it' step — an explicit check that the reviewed output meets the standard before you move on.
- Keep every reviewed output and prompt in one project folder (Tempo-2.0-Playbook) so your playbook stays together and consistent.

## Topic 01 — Getting Started with Generative AI for Agile (25%)

Introduction to Agile, Scrum and generative AI · Setting up AI tools for agile project management · Effective prompting for agile tasks · Responsible and human-in-the-loop use of AI

### Key concepts

- Agile and Scrum in one view — Agile is a way of delivering value in small, inspect-and-adapt increments; Scrum is the most common framework, with roles (Product Owner, Scrum Master, Developers), events (sprint planning, daily stand-up, review, retrospective) and artifacts (product backlog, sprint backlog, increment).
- Where generative AI fits — a generative AI assistant is a drafting, summarising and analysis partner across the whole Scrum cycle; it speeds up the writing and thinking work (stories, plans, summaries, reports) so the team spends more time on judgement, collaboration and delivery.
- The three assistants — ChatGPT, Claude and Gemini are general-purpose chat assistants that all take a text prompt and return a draft; they differ in interface, context length and integrations, but the prompting and review skills you learn here transfer across all three.
- AI is a co-pilot, not the pilot — the AI drafts and suggests; the Product Owner, Scrum Master and team still own every decision, estimate and commitment. Nothing goes into the backlog, plan or report without a human reviewing it.
- Setting up your AI workspace — you sign in to an assistant, learn to start a focused chat per task, paste in the relevant context (the project brief, backlog, notes), and keep your AI work organised alongside your agile tooling.
- Prompting is the core skill — a good agile prompt gives the AI a role, the context, the exact task, the format you want back and any constraints; a vague ask gives a vague draft, a structured ask gives a usable one.
- A reusable prompt library — the same agile tasks recur every sprint (write stories, groom the backlog, summarise the stand-up, draft the report), so you save your best prompts as reusable templates the whole team can run.
- Human-in-the-loop review — every AI output is a first draft to be checked: is it accurate, does it fit our context, is anything invented (a 'hallucination'), is it fair and clear? You review, correct and own the result.
- Responsible and safe use — you avoid pasting confidential customer data, credentials or personal information into a public assistant, you check the facts and figures the AI states, and you are transparent with the team about what was AI-assisted.

### Lab 1 — Set Up Your AI Toolkit for Agile Project Management

Learning outcome: Sign in to ChatGPT, Claude and Gemini, give the same agile task to each, compare their drafts, and set up your Tempo 2.0 workspace and the human-in-the-loop review habit that every later lab depends on.

**Goal:** This lab gets you comfortable with the tools before any real planning begins. You open ChatGPT, Claude and Gemini in your browser, sign in to each, and read the supplied Tempo 2.0 project brief so you know the product you will be working on. You give all three assistants the same simple agile task, compare how they respond, and see that they work the same way from your point of view — a prompt in, a draft out. You then establish the two habits that run through the whole course: keeping an organised project folder for your playbook, and treating every AI answer as a first draft you review rather than a fact you accept. **BUILDING BLOCK** — what you create in this lab becomes a section of your Tempo 2.0 Agile AI Playbook, the connected project you assemble across all 12 labs.

## What you'll build

A working AI toolkit — ChatGPT, Claude and Gemini signed in and tested — plus a Tempo-2.0-Playbook project folder, a short written note on how the three assistants differed on the same task, and a clear grasp of the prompt-review-own workflow. (Tools: ChatGPT ([chat.openai.com](https://chat.openai.com)), Claude ([claude.ai](https://claude.ai)), Gemini ([gemini.google.com](https://gemini.google.com)), the Tempo 2.0 project brief, a project folder, the prompt-review-own workflow.)

## Step-by-step

1. Create a folder on your machine called 'Tempo-2.0-Playbook' so every reviewed output and prompt you make across the course stays together. Open the supplied Tempo 2.0 project brief ([labs/reference-pack/](#)) and skim the product vision, the team and the sample inputs so you know what you are working on.
2. Open all three assistants in separate browser tabs — [chat.openai.com](https://chat.openai.com) (ChatGPT), [claude.ai](https://claude.ai) (Claude) and [gemini.google.com](https://gemini.google.com) (Gemini) — and sign in to each. Send each a simple 'hello' to confirm it responds.
3. Give the SAME agile task to each assistant so you can compare them. Paste the prompt below into ChatGPT, then into Claude, then into Gemini.

You are an Agile coach. In plain language, explain to a new Scrum team the purpose of the four main Scrum events – sprint planning, the daily stand-up, the sprint review and the retrospective – in one short sentence each. Keep it under 120 words total.

4. Read the three answers side by side. Note in one or two lines where they differ — length, tone, clarity, formatting. This is your first evidence that the tool matters less than the prompt and your review.
5. Now feel how context changes the answer. Copy the product-vision paragraph from the Tempo 2.0 brief and paste it into any one assistant with the prompt below.

Here is the product vision for our app, Tempo:

<PASTE THE TEMPO 2.0 PRODUCT VISION HERE>

In three bullet points, summarise what this product is for and who it serves, using only the information above. Do not invent features that are not stated.

6. Deliberately test the review habit: check the assistant's summary against the brief. Did it stick to what the brief actually says, or did it add something that is not there? Note anything invented — this is exactly the human-in-the-loop check you will do on every output.
7. Write a short 'how I'll work' note for yourself and save it in your Tempo-2.0-Playbook folder: which assistant(s) you will use, and the three-step rule you will follow every time — prompt, review, own. This note opens your playbook.

### Test it

You have ChatGPT, Claude and Gemini signed in and responding, a Tempo-2.0-Playbook folder created, a written note comparing how the three tools answered the same agile task, and you have practised the prompt-review-own workflow — including catching whether a summary stayed true to the brief.

**Note:** Full commands and screenshots are in labs/lab-01-\*.md. Use only projects, data and notes you are authorised to use. Do not paste confidential customer data, personal information, credentials or proprietary code into a public AI assistant. Use the supplied Tempo 2.0 project brief rather than real client material, treat every AI output as a first draft to be reviewed and fact-checked, and be transparent with your team about what was AI-assisted before it goes into a real backlog, plan, report or decision.

## Lab 2 — Write Effective Prompts and Build an Agile Prompt Library

Learning outcome: Learn the role-context-task-format-constraints structure for agile prompts, refine a weak prompt into a strong one, and save a reusable agile prompt library you will use for the rest of the course.

Goal: A good AI draft starts with a good prompt, not a lucky one. In this lab you learn a simple, reliable structure for agile prompts — give the AI a role, the context, the exact task, the output format you want, and any constraints — and you see how each part changes the result. You take a deliberately weak, vague prompt and improve it one part at a time until it produces a genuinely usable draft. Then, because the same agile tasks recur every sprint, you save your best prompts as a reusable prompt library with clearly marked slots, so you (and your team) never start from a blank box again. BUILDING BLOCK — what you create in this lab becomes a section of your Tempo 2.0 Agile AI Playbook, the connected project you assemble across all 12 labs.

### What you'll build

A reusable agile prompt library saved in your project folder — structured, slot-based prompt templates for the recurring agile tasks (user stories, backlog grooming, stand-up summary, status report, retrospective) — plus a before/after example proving a structured prompt beats a vague one. (Tools: Any assistant (ChatGPT, Claude or Gemini), the role-context-task-format-constraints prompt structure, single-change prompt edits, a saved slot-based prompt library.)

## Step-by-step

1. Start with a deliberately weak prompt so you can feel the difference. In any assistant, send: 'Write some user stories for a habit app.' Read the vague, generic result and keep it to compare against.
2. Now rebuild the same request with structure. Send the prompt below and compare the result with the weak one — notice how much more usable it is.

You are an experienced Agile Product Owner. Context: we are building Tempo 2.0, a habit-tracking app; a key new feature is shared team habits, where a group tracks a habit together. Task: write three user stories for this feature. Format: use the form 'As a <role>, I want <goal>, so that <benefit>', one per line. Constraints: keep each to one sentence, make them independent, and focus on the user's value, not the technical solution.

3. Change exactly one part and regenerate, so you can attribute the change. First change the ROLE (for example to 'a strict Agile coach who insists on INVEST'); read how the emphasis shifts.
4. Now change only the FORMAT — ask for the same three stories as a markdown table with columns Role, Goal, Benefit, Priority. Notice format is independent of content.

Re-present exactly those three user stories as a markdown table with the columns: Role | Goal | Benefit | Priority (High/Medium/Low). Do not change the stories' wording.

5. Now change only the CONSTRAINTS — add 'each story must be small enough to finish in one sprint, and add one acceptance criterion per story'. See how constraints tighten quality without you rewriting the whole prompt.
6. Extract the pattern into a reusable template. Ask the assistant to help, then save the result. Paste the prompt below.

Turn the effective prompt we just refined into a reusable template with clearly marked slots: [ROLE], [CONTEXT], [TASK], [FORMAT], [CONSTRAINTS]. Then create four more templates in the same slot style for these recurring agile tasks: grooming a backlog, summarising a daily stand-up from raw notes, writing a sprint status report for stakeholders, and running a retrospective. Present all five as a clean, copy-ready prompt library.

7. Review and save. Read each template critically — would it work on a real task? Fix anything weak, then save the five-template set as 'agile-prompt-library' in your Tempo-2.0-Playbook folder. Keep the weak-vs-structured example next to it as a reminder of why structure matters.

## Test it

You have a saved agile prompt library of at least five slot-based templates (user stories, backlog grooming, stand-up summary, status report, retrospective) built on the role-context-task-format-constraints structure, and a before/after example that shows a structured prompt clearly beats a vague one.

**Note:** Full commands and screenshots are in labs/lab-02-\*.md. Use only projects, data and notes you are authorised to use. Do not paste confidential customer data, personal information, credentials or proprietary code into a public AI assistant. Use the supplied Tempo 2.0 project brief rather than real client material, treat every AI output as a first draft to be reviewed and fact-checked, and be transparent with your team about what was AI-assisted before it goes into a real backlog, plan, report or decision.

### Lab 3 — Apply Responsible, Human-in-the-Loop AI Practices

Learning outcome: Build a responsible-AI checklist for agile work and apply it — catching an invented 'fact', rewriting a prompt to protect sensitive data, and confirming a draft is fair, clear and owned by a human.

Goal: Using AI in a real delivery brings real responsibilities. In this lab you build and apply the human-in-the-loop discipline that keeps AI use safe. You deliberately prompt an assistant into stating something confidently wrong (a 'hallucination') and practise catching and correcting it. You learn what never to paste into a public assistant — customer data, personal information, credentials, proprietary code — and rewrite a risky prompt to remove it. You check a draft for fairness and clarity, and you decide how your team will be transparent about what was AI-assisted. The output is a short, reusable responsible-AI checklist you keep visible for the rest of the course. **BUILDING BLOCK** — what you create in this lab becomes a section of your Tempo 2.0 Agile AI Playbook, the connected project you assemble across all 12 labs.

#### What you'll build

A one-page responsible-AI checklist for agile work (fact-check, protect data, be fair and clear, disclose AI use, keep a human owner) plus worked examples — a caught-and-corrected hallucination and a risky prompt rewritten to remove sensitive data. (Tools: Any assistant, fact-checking against the brief, data-safety rewriting, a fairness and clarity review, a transparency and ownership decision.)

#### Step-by-step

1. See a hallucination for yourself. Ask an assistant a specific, checkable question it is likely to get wrong or invent, for example the prompt below. Read how confident the answer sounds.

According to the official Scrum Guide, exactly how many minutes long must a daily stand-up be, and on which page is that stated? Give the exact page number.

2. Fact-check it. The Scrum Guide time-boxes the daily stand-up to 15 minutes but does not mandate an exact length by page number — so any confident 'page number' is invented. Note this as your first checklist rule: verify specific facts, figures and citations before you trust them.
3. Practise data safety. Look at this risky prompt a busy Scrum Master might send: 'Here is our customer list with emails and phone numbers <pasted>, and our AWS

admin password is <pasted> — write release notes.' Identify everything in it that must never go into a public assistant.

4. Rewrite it safely. Redraft the same request so it gets the job done with no sensitive data, using the prompt below as your safe version.

You are a release manager. Write concise release notes for a mobile app update that adds shared team habits, a smarter reminder engine and an insights dashboard, and fixes several reminder-reliability bugs. Use only this description – do not ask for or include any customer data, personal information or credentials. Format: a short 'New', 'Improved' and 'Fixed' list.

5. Check for fairness and clarity. Take any draft the AI has written for you so far and ask it to review itself with the prompt below; then read its answer critically — you, not the AI, are the judge.

Review the text above for clarity and fairness: is anything ambiguous, biased, exclusionary or likely to be misread by a non-technical stakeholder? List specific issues and suggest plain, neutral rewrites. Do not change the meaning.

6. Decide on transparency and ownership. Write one line on how your team will note when something was AI-assisted (for example a small 'AI-assisted, reviewed by <name>' tag on drafts), and confirm the rule that a named human owns and approves every output before it is used.

7. Assemble your responsible-AI checklist. Combine what you practised into a short, reusable checklist — fact-check specifics; never paste sensitive data; review for fairness and clarity; disclose AI assistance; a human owns and approves — and save it as 'responsible-ai-checklist' in your Tempo-2.0-Playbook. Keep it open beside you for every remaining lab.

## Test it

You have a saved one-page responsible-AI checklist, and you have applied it in practice: you caught and corrected an invented 'fact', you identified and removed sensitive data by rewriting a risky prompt safely, and you reviewed a draft for fairness, clarity, disclosure and human ownership.

**Note:** Full commands and screenshots are in labs/lab-03-\*.md. Use only projects, data and notes you are authorised to use. Do not paste confidential customer data, personal information, credentials or proprietary code into a public AI assistant. Use the supplied Tempo 2.0 project brief rather than real client material, treat every AI output as a first draft to be reviewed and fact-checked, and be transparent with your team about what was AI-assisted before it goes into a real backlog, plan, report or decision.

## Topic 02 — AI for Backlogs, User Stories and Planning (25%)

Drafting and refining user stories and acceptance criteria · Grooming and prioritising the backlog with AI · Sprint planning and estimation support · Generating roadmaps and release plans

### Key concepts

- User stories with AI — the AI turns a rough feature idea into a well-formed user story in the 'As a <role>, I want <goal>, so that <benefit>' form, and you refine it so it reflects the real user and real value.
- INVEST-quality stories — good stories are Independent, Negotiable, Valuable, Estimable, Small and Testable; you prompt the AI to draft against INVEST and to flag stories that are too big or vague to estimate.
- Acceptance criteria — the AI drafts clear, testable acceptance criteria (often in Given/When/Then form) that define 'done' for a story, which you then check for gaps, edge cases and testability.
- Grooming the backlog — the AI helps refine, split, merge and de-duplicate backlog items, clarify wording and add missing detail, turning a messy raw list into a groomed, ready backlog.
- Prioritising with AI — the AI applies frameworks such as MoSCoW (Must / Should / Could / Won't) and value-versus-effort to suggest a priority order and explain its reasoning; the Product Owner makes the final call.
- Sprint planning support — given the team's capacity and the top of the backlog, the AI proposes a realistic sprint goal and a candidate sprint backlog, and helps break stories into tasks.
- Estimation support — the AI suggests relative story-point estimates and surfaces the assumptions and risks behind each, giving the team a sensible starting point for planning-poker discussion rather than a number to accept blindly.
- Roadmaps and release plans — the AI drafts a multi-sprint roadmap and a release plan from the prioritised backlog, grouping work into themes and releases and making dependencies and milestones explicit.
- Keeping ownership — AI accelerates the drafting, but the team still negotiates scope, commits to the sprint and owns the estimates; the AI's numbers are inputs to the conversation, never the decision.

### Lab 4 — Draft and Refine User Stories and Acceptance Criteria

Learning outcome: Turn raw Tempo 2.0 feature ideas into INVEST-quality user stories with AI, then draft and refine testable acceptance criteria, reviewing every one so it reflects the real user and real value.

Goal: Planning starts with well-formed stories. In this lab you take the rough feature ideas from the Tempo 2.0 brief and use an assistant to draft them as proper user stories in the 'As a <role>, I want <goal>, so that <benefit>' form. You prompt the AI to check its own stories against INVEST and to flag any that are too big or vague to estimate, then

split those with its help. For your key stories you draft acceptance criteria in Given/When/Then form and review them for gaps, edge cases and testability. Throughout, you apply your responsible-AI checklist — the AI drafts, you decide what is right for the real user. BUILDING BLOCK — what you create in this lab becomes a section of your Tempo 2.0 Agile AI Playbook, the connected project you assemble across all 12 labs.

## What you'll build

A set of INVEST-checked Tempo 2.0 user stories (with over-large ones split) and, for the key stories, testable Given/When/Then acceptance criteria — all reviewed by you and saved as the stories section of your playbook. (Tools: Any assistant, the user-story template from your prompt library, the INVEST checklist, story splitting, Given/When/Then acceptance criteria.)

## Step-by-step

1. Open the Tempo 2.0 brief and copy the raw feature ideas for the 2.0 release (shared team habits, the smarter reminder engine, the insights dashboard, and the listed bug fixes). Open your agile prompt library and grab your user-story template.
2. Draft the first batch of stories. Paste the prompt below, filling the context slot with the raw ideas.

You are an experienced Agile Product Owner for Tempo, a habit-tracking app.  
Context: here are the raw feature ideas for our 2.0 release:

<PASTE THE RAW FEATURE IDEAS>

Task: write clear user stories in the form 'As a <role>, I want <goal>, so that <benefit>'. Format: group them under the three feature themes (shared team habits, smarter reminders, insights dashboard). Constraints: focus on user value, keep each story to one sentence, and cover the main users (an individual user, a team member, a team admin).

3. Quality-check against INVEST. Ask the AI to grade its own stories and flag the weak ones, using the prompt below, then read its judgement critically.

Review each story above against INVEST (Independent, Negotiable, Valuable, Estimable, Small, Testable). Present a table: Story | INVEST issues | Suggested fix. Be strict – flag any story that is too big or vague to estimate in one sprint.

4. Split the over-large stories. Take one story the AI flagged as too big and ask it to split the work into smaller, independently valuable stories.

Split this user story into 2-4 smaller stories that are each independently valuable, small enough to finish in one sprint, and testable. Keep the same 'As a / I want / so that' form:

<PASTE THE OVER-LARGE STORY>

5. Draft acceptance criteria for your key stories. Choose the three most important stories and generate testable criteria with the prompt below.

For each of the three user stories below, write acceptance criteria in Given/When/Then form that define exactly when the story is done. Include at least one edge case or error condition per story. Keep each criterion testable and unambiguous.

<PASTE THE THREE KEY STORIES>

6. Review the acceptance criteria as a human. For each story ask: does 'done' really mean done here? Is a realistic edge case missing (offline, permissions, an empty state, a shared-habit conflict)? Add or correct criteria yourself where the AI missed something.
7. Save the stories section of your playbook: the INVEST-checked stories grouped by theme, the split of the over-large story, and the reviewed Given/When/Then acceptance criteria for the key stories, in your Tempo-2.0-Playbook folder.

### Test it

You have a reviewed set of Tempo 2.0 user stories in proper form, checked against INVEST with at least one over-large story split into smaller ones, and testable Given/When/Then acceptance criteria (including edge cases) for your key stories — all human-reviewed and saved to your playbook.

**Note:** Full commands and screenshots are in labs/lab-04-\*.md. Use only projects, data and notes you are authorised to use. Do not paste confidential customer data, personal information, credentials or proprietary code into a public AI assistant. Use the supplied Tempo 2.0 project brief rather than real client material, treat every AI output as a first draft to be reviewed and fact-checked, and be transparent with your team about what was AI-assisted before it goes into a real backlog, plan, report or decision.

## Lab 5 — Groom and Prioritise the Product Backlog with AI

Learning outcome: Use AI to groom a messy raw backlog — clarifying, splitting, merging and de-duplicating items — then prioritise it with MoSCoW and value-versus-effort, keeping the final ordering a human decision.

Goal: A raw idea list is not a backlog. In this lab you turn the Tempo 2.0 stories and remaining raw items into a groomed, ordered product backlog. You use an assistant to clean the list — clarify vague wording, split items that are really several, merge duplicates, and fill obvious gaps — so every item is ready. Then you prioritise: the AI applies MoSCoW (Must / Should / Could / Won't) and a value-versus-effort view and explains its reasoning, and you, in the Product Owner's seat, make the final call and adjust the order. The result is a ready, prioritised backlog you can plan a sprint from. BUILDING BLOCK — what you create in this lab becomes a section of your Tempo 2.0 Agile AI Playbook, the connected project you assemble across all 12 labs.

## What you'll build

A groomed, prioritised Tempo 2.0 product backlog — cleaned and de-duplicated, each item tagged with a MoSCoW category and a value/effort view, ordered for planning, with your Product-Owner adjustments applied. (Tools: Any assistant, the backlog-grooming template from your prompt library, MoSCoW prioritisation, value-versus-effort analysis, Product-Owner review.)

## Step-by-step

1. Assemble your raw backlog: your Lab 4 stories plus any remaining raw ideas and bug fixes from the Tempo 2.0 brief that are not yet stories. Paste them into any assistant as one messy list so you can groom it.
2. Groom the list. Use the prompt below to clarify, split, merge and de-duplicate.

You are helping a Product Owner groom a product backlog. Here is our raw Tempo 2.0 backlog:

<PASTE THE RAW BACKLOG>

Task: return a cleaned, ready backlog. Clarify any vague item, split any item that is really several, merge duplicates, and flag anything missing important detail. Present it as a numbered list with a one-line description per item. Do not invent new features that are not implied by the list.

3. Review the grooming. Check the AI did not quietly drop or invent items. Confirm each split and merge makes sense to you; undo any you disagree with. Grooming is a judgement call, not an automatic one.
4. Prioritise with MoSCoW. Ask the AI to categorise and explain, using the prompt below.

Prioritise the groomed backlog using MoSCoW (Must have, Should have, Could have, Won't have this release). For the Tempo 2.0 release, our goal is to ship shared team habits and reliable reminders first. Present a table: Item | MoSCoW | One-line reason. Then list the 'Must have' items in the order you would build them and explain the ordering.

5. Add a value-versus-effort view. Ask the AI to estimate relative value and effort so you can spot quick wins and expensive extras.

For each backlog item, add a rough Value (High/Medium/Low) and Effort (High/Medium/Low) rating and mark any that are High-value / Low-effort as 'quick win'. Present as a table and briefly note which items look like poor value for their effort.

6. Make the Product Owner's call. Reconcile MoSCoW and value/effort into one ordered backlog. Move at least two items yourself against the AI's suggestion where you disagree, and write one line explaining each override — this is the human decision the AI supports but does not make.

7. Save the prioritised backlog section of your playbook: the groomed items, their MoSCoW and value/effort tags, the final ordering, and your override notes, in your Tempo-2.0-Playbook folder.

### Test it

You have a groomed, de-duplicated Tempo 2.0 backlog with every item tagged by MoSCoW and value/effort, ordered ready for planning, and you have made at least two Product-Owner overrides against the AI's suggestion with reasons — a backlog you own, not one the AI decided.

**Note:** Full commands and screenshots are in labs/lab-05-\*.md. Use only projects, data and notes you are authorised to use. Do not paste confidential customer data, personal information, credentials or proprietary code into a public AI assistant. Use the supplied Tempo 2.0 project brief rather than real client material, treat every AI output as a first draft to be reviewed and fact-checked, and be transparent with your team about what was AI-assisted before it goes into a real backlog, plan, report or decision.

## Lab 6 — Plan the Sprint, Estimate, and Build the Release Roadmap

Learning outcome: Use AI to propose a sprint goal and candidate sprint backlog from the team's capacity, get relative estimates with their assumptions surfaced, and draft a multi-sprint roadmap and release plan — with the team owning every commitment.

Goal: With a prioritised backlog you can plan. In this lab you use an assistant to support sprint planning end to end. Given the team's capacity and the top of the backlog, the AI proposes a realistic sprint goal and a candidate sprint backlog and helps break the top stories into tasks. It suggests relative story-point estimates and — crucially — surfaces the assumptions and risks behind each, giving the team a starting point for a planning-poker conversation rather than numbers to accept. Finally it drafts a multi-sprint roadmap and a release plan from the backlog, making themes, dependencies and milestones explicit. You keep ownership throughout: the AI's numbers are inputs, the team's commitment is the decision. This lab completes Day 1. BUILDING BLOCK — what you create in this lab becomes a section of your Tempo 2.0 Agile AI Playbook, the connected project you assemble across all 12 labs.

### What you'll build

A first-sprint plan for Tempo 2.0 (a sprint goal, a candidate sprint backlog broken into tasks), relative estimates with their assumptions and risks surfaced, and a multi-sprint roadmap and release plan — all reviewed and adjusted by you. (Tools: Any assistant, the prioritised backlog from Lab 5, team capacity, sprint-goal and task breakdown, story-point estimation support, roadmap and release planning.)

### Step-by-step

1. Gather your inputs: your prioritised backlog from Lab 5 and the team's capacity from the brief (team size, sprint length, and any planned leave). Note the sprint length is two weeks.
2. Propose a sprint goal and candidate sprint backlog. Paste the prompt below with your inputs.

You are an Agile delivery assistant. Context: here is our prioritised Tempo 2.0 backlog:

<PASTE THE PRIORITISED BACKLOG>

Our team capacity for a 2-week sprint is <TEAM CAPACITY>. Task: propose one clear sprint goal for Sprint 1 and a candidate sprint backlog of items that fit the capacity and serve that goal. Format: state the sprint goal in one sentence, then list the selected items. Constraints: prefer the 'Must have' items and a coherent goal over cramming in unrelated work.

3. Break the top stories into tasks. Ask the AI to decompose the candidate sprint backlog into concrete tasks.

Break each item in the candidate sprint backlog into the concrete tasks needed to deliver it (design, build, test, review, etc.). Present as a checklist grouped by story. Keep tasks small enough to finish in a day or two.

4. Get estimation support with assumptions surfaced. Use the prompt below — the assumptions matter more than the numbers.

Suggest a relative story-point estimate (using the sequence 1, 2, 3, 5, 8, 13) for each item in the candidate sprint backlog. For every estimate, state the key assumption and the main risk behind it in one line. Make clear these are a starting point for the team's planning-poker discussion, not final numbers.

5. Run the human estimation check. Pick two items where you disagree with the AI's story points and write your own estimate and reasoning. This mirrors the planning-poker conversation the team would have — the AI opens it, the team settles it.
6. Draft the roadmap and release plan. Generate a higher-level view with the prompt below.

From the full prioritised backlog, draft a multi-sprint roadmap for Tempo 2.0 across the next 4 sprints, grouping work into themes (shared team habits, smarter reminders, insights dashboard, fixes and polish). Then draft a release plan showing which themes ship in which release, and list the key dependencies and milestones. Present the roadmap as a table (Sprint | Theme | Main items) and the release plan as a short bulleted plan. State any assumption you make.

7. Review and save. Sanity-check the roadmap against reality — is the sequence sensible, are dependencies right, is anything over-committed? Adjust it yourself, then save the sprint plan, estimates and roadmap as the planning section of your Tempo-2.0-Playbook. Day 1 is complete: you have a toolkit, a prioritised backlog and a plan.

## Test it

You have a reviewed Sprint 1 plan (a one-sentence sprint goal and a capacity-fit sprint backlog broken into tasks), story-point estimates with their assumptions and risks surfaced and at least two you re-estimated yourself, and a multi-sprint roadmap and release plan with dependencies and milestones — all saved to your playbook.

**Note:** Full commands and screenshots are in labs/lab-06-\*.md. Use only projects, data and notes you are authorised to use. Do not paste confidential customer data, personal information, credentials or proprietary code into a public AI assistant. Use the supplied Tempo 2.0 project brief rather than real client material, treat every AI output as a first draft to be reviewed and fact-checked, and be transparent with your team about what was AI-assisted before it goes into a real backlog, plan, report or decision.

### Topic 03 — AI for Sprints, Standups and Delivery (25%)

Summarising stand-ups and tracking progress · Identifying risks, blockers and dependencies with AI · Assisting documentation and communication · Supporting quality, testing and delivery

#### Key concepts

- Summarising stand-ups — the AI turns raw daily-stand-up notes into a clear summary of what was done, what is planned and what is blocked, so the team and stakeholders get a consistent update without extra writing.
- Tracking progress — from stand-up notes and board status, the AI helps describe sprint progress against the goal, highlight what has moved and what is stuck, and draft a plain-language burndown narrative.
- Surfacing risks and blockers — the AI reads the team's notes and backlog and flags likely risks, blockers and impediments early, so the Scrum Master can act before they derail the sprint.
- Mapping dependencies — the AI identifies dependencies between stories, teams and external parties, and helps sequence work so blocked items are unblocked in the right order.
- Mitigation and escalation — for each risk or blocker the AI suggests mitigation options and a clear escalation path, which the Scrum Master reviews and turns into real actions and owners.
- Assisting documentation — the AI drafts the routine sprint documentation (definition of done notes, decision logs, release notes, meeting minutes) so it is written consistently and kept up to date with far less effort.
- Assisting communication — the AI tailors the same update for different audiences — a technical note for the team, a plain summary for stakeholders, a short message for leadership — keeping tone and detail appropriate to each.

- Supporting quality and testing — the AI drafts test scenarios and checklists from acceptance criteria, suggests edge cases, and helps review whether a story truly meets its definition of done.
- Human judgement in delivery — the AI drafts and flags, but the Scrum Master and team validate every risk, dependency and test; the AI widens what you notice, it does not replace the team's call.

## Lab 7 — Summarise Stand-ups and Track Sprint Progress

Learning outcome: Turn raw daily stand-up notes into a clear, consistent summary with AI, then track progress against the sprint goal and draft a plain-language burndown narrative — keeping the team's read of reality in the loop.

Goal: Once the sprint is running, a lot of the Scrum Master's time goes into writing updates. In this lab you hand that drafting to an assistant. You take the supplied raw daily stand-up notes for Tempo 2.0 — terse, messy, in everyone's own words — and prompt the AI to turn them into a clean summary of what was done, what is planned and what is blocked. You then track progress against the sprint goal: you ask the AI to describe what has moved and what is stuck, and to write a plain-language burndown narrative from the remaining work. You review every summary against what you actually know, because the AI can only work from the notes it is given. **BUILDING BLOCK** — what you create in this lab becomes a section of your Tempo 2.0 Agile AI Playbook, the connected project you assemble across all 12 labs.

### What you'll build

A clean daily stand-up summary generated from raw notes, a progress-against-goal update, and a plain-language burndown narrative for the Tempo 2.0 sprint — all reviewed and saved as the tracking section of your playbook. (Tools: Any assistant, the stand-up-summary template from your prompt library, the raw stand-up notes, progress-against-goal tracking, a burndown narrative.)

### Step-by-step

1. Open the supplied raw daily stand-up notes for the Tempo 2.0 sprint (labs/reference-pack/) — several days of terse, informal updates from the team. Grab your stand-up-summary template from your prompt library.
2. Summarise one day's stand-up. Paste the prompt below with a single day's raw notes.

You are a Scrum Master assistant. Here are today's raw daily stand-up notes for our Tempo 2.0 sprint:

<PASTE ONE DAY'S RAW NOTES>

Task: summarise them clearly under three headings – Done since yesterday, Planned today, Blockers. Format: short bullet points. Constraints: use only what the notes say, keep each person's items attributed, and list any blocker separately so nothing is buried.

3. Review the summary against reality. Check it captured every blocker and did not invent progress. If a note was ambiguous, decide what it really meant — the AI cannot know, you can.
4. Track progress against the sprint goal. Feed several days of notes and the sprint goal together with the prompt below.

Here is our Sprint 1 goal:

<PASTE THE SPRINT GOAL>

and here are the stand-up summaries for the last few days:

<PASTE THE DAILY SUMMARIES>

Task: describe our progress toward the sprint goal – what has clearly moved forward, what is stuck, and whether the goal still looks achievable this sprint. Be honest and specific; do not sugar-coat. Flag anything that looks at risk.

5. Draft a burndown narrative. Ask the AI to describe the remaining work in plain language, using the prompt below.

Given that the sprint started with <TOTAL STORY POINTS> points and roughly <REMAINING POINTS> remain with <DAYS LEFT> days left, write a short plain-language 'burndown' narrative a non-technical stakeholder could understand: are we ahead, on track or behind, and what would need to happen to finish on plan? State the assumption behind your read.

6. Apply your judgement. Compare the AI's 'on track / behind' read with your own sense of the sprint. Where they differ, write the truer version yourself. A burndown narrative is only useful if it is honest.
7. Save the tracking section of your playbook: the daily stand-up summary, the progress-against-goal update and the reviewed burndown narrative, in your Tempo-2.0-Playbook folder.

## Test it

You have turned raw stand-up notes into a clean, attributed summary under Done/Planned/Blockers, produced an honest progress-against-goal update and a plain-language burndown narrative for the Tempo 2.0 sprint, and reviewed each against what you actually know — all saved to your playbook.

**Note:** Full commands and screenshots are in labs/lab-07-\*.md. Use only projects, data and notes you are authorised to use. Do not paste confidential customer data, personal information, credentials or proprietary code into a public AI assistant. Use the supplied Tempo 2.0 project brief rather than real client material, treat every AI output as a first draft to be reviewed and fact-checked, and be transparent with your team about what was AI-assisted before it goes into a real backlog, plan, report or decision.

## Lab 8 — Identify Risks, Blockers and Dependencies with AI

Learning outcome: Use AI to read the team's notes and backlog and surface likely risks, blockers and dependencies early, map how the work depends on itself and on outside parties, and turn each into a mitigation and escalation the Scrum Master owns.

Goal: A big part of the Scrum Master's value is seeing trouble early. In this lab you use an assistant to widen what you notice. You give the AI the sprint's stand-up notes, backlog and plan, and prompt it to flag likely risks, blockers and impediments — including ones the team has not named yet. You ask it to map dependencies between stories, between the team and other teams, and on external parties, and to suggest a sequence that unblocks work in the right order. For each risk or blocker it drafts mitigation options and a clear escalation path. You then review every item — the AI can over- or under-state risk — and turn the real ones into a risk log with owners and actions. BUILDING BLOCK — what you create in this lab becomes a section of your Tempo 2.0 Agile AI Playbook, the connected project you assemble across all 12 labs.

### What you'll build

A reviewed Tempo 2.0 risk, blocker and dependency log — likely risks and impediments surfaced from the notes, a dependency map with a suggested unblocking sequence, and a mitigation and escalation path with an owner for each real item. (Tools: Any assistant, the sprint notes and backlog, risk and blocker identification, dependency mapping, mitigation and escalation planning, a Scrum-Master review.)

### Step-by-step

1. Gather the evidence: your stand-up summaries and burndown from Lab 7, your sprint plan from Lab 6, and the backlog. Paste the relevant parts into any assistant.
2. Surface risks and blockers. Use the prompt below to have the AI flag trouble early.

You are an experienced Scrum Master. Based on this Tempo 2.0 sprint information:

<PASTE STAND-UP SUMMARIES, PLAN AND BACKLOG>

Task: identify the likely risks, blockers and impediments to finishing this sprint – including ones the team has not explicitly named. Format: a table with Risk/Blocker | Why it matters | Likelihood (H/M/L) | Impact (H/M/L). Be specific to this sprint; do not list generic project risks.

3. Map dependencies. Ask the AI to make the dependencies explicit with the prompt below.

From the same sprint information, map the dependencies: which stories or tasks depend on another being done first, which depend on another team or an external party (for example an app-store review, a third-party reminder/push service, or a design sign-off), and where a dependency could block the sprint goal. Present as a list of 'A depends on B because...' statements, then suggest an order of work that unblocks things in the right sequence.

4. Sanity-check the AI's read. Go through its risks and dependencies and mark each as real, over-stated or missed. Add at least one risk or dependency you know about from context that the AI did not surface. The AI widens your view; it does not replace it.

5. Plan mitigations and escalations. For the real risks and blockers, generate options with the prompt below.

For each of these confirmed risks and blockers:

<PASTE THE CONFIRMED LIST>

suggest one or two practical mitigation options and a clear escalation path (who to raise it to and when). Format: Risk/Blocker | Mitigation | Escalation. Keep the actions concrete and doable within a sprint.

6. Assign owners and decide actions. As the Scrum Master, put a named owner and a next action against each real risk and blocker — the AI proposes, you commit. Remove anything that is noise so the log stays trusted and short.

7. Save the risk, blocker and dependency log as a section of your Tempo-2.0-Playbook folder: the confirmed items with likelihood/impact, the dependency map and unblocking order, and each item's mitigation, escalation, owner and next action.

### Test it

You have a reviewed Tempo 2.0 risk, blocker and dependency log — likely items surfaced early, a dependency map with a sensible unblocking order, at least one risk you added from your own knowledge, and a mitigation, escalation, owner and next action for every real item — saved to your playbook.

**Note:** Full commands and screenshots are in labs/lab-08-\*.md. Use only projects, data and notes you are authorised to use. Do not paste confidential customer data, personal information, credentials or proprietary code into a public AI assistant. Use the supplied Tempo 2.0 project brief rather than real client material, treat every AI output as a first draft to be reviewed and fact-checked, and be transparent with your team about what was AI-assisted before it goes into a real backlog, plan, report or decision.

## Lab 9 — Assist Documentation, Communication and Testing

Learning outcome: Use AI to draft the sprint's routine documentation, tailor the same update for different audiences, and generate test scenarios and a definition-of-done check from acceptance criteria — reviewing each so quality stays the team's call.

Goal: Documentation, communication and testing quietly consume a sprint. In this lab you use an assistant to lift that load across all three. For documentation, you draft release notes, a decision log entry and meeting minutes so they are written consistently and kept current. For communication, you take one sprint update and have the AI re-pitch it for three audiences — the team, stakeholders and leadership — keeping tone

and detail right for each. For quality, you turn your Lab 4 acceptance criteria into test scenarios and a checklist, ask the AI to suggest edge cases, and use it to check whether a story truly meets its definition of done. You review everything: the AI drafts, the team decides what 'done' and 'good enough' mean. **BUILDING BLOCK** — what you create in this lab becomes a section of your Tempo 2.0 Agile AI Playbook, the connected project you assemble across all 12 labs.

### What you'll build

AI-assisted sprint documentation (release notes, a decision log entry, meeting minutes), one update re-pitched for the team, stakeholders and leadership, and test scenarios plus a definition-of-done check generated from your acceptance criteria — all reviewed and saved. (Tools: Any assistant, documentation drafting, audience-tailored communication, test-scenario generation from acceptance criteria, a definition-of-done check.)

### Step-by-step

1. Draft the routine documentation. Using what the sprint has delivered so far, generate release notes with the prompt below, then repeat the idea for a decision-log entry and short meeting minutes.

You are a delivery documentation assistant. Based on the work completed in our Tempo 2.0 sprint so far:

<PASTE THE COMPLETED ITEMS>

Task: write concise release notes under 'New', 'Improved' and 'Fixed'. Then, separately, draft a one-paragraph decision-log entry template and a short meeting-minutes template we can reuse. Constraints: use only the information given, and mark anything uncertain as '[to confirm]' rather than inventing it.

2. Review the docs for invented detail. Check the release notes claim only what was actually done, and that every '[to confirm]' is a real gap for a human to fill. Correct anything overstated.
3. Tailor communication for three audiences. Take one progress update and re-pitch it with the prompt below.

Here is a sprint progress update:

<PASTE YOUR PROGRESS UPDATE FROM LAB 7>

Rewrite it for three audiences: (1) the development team – technical, detailed, honest about blockers; (2) stakeholders – plain language, focused on value and dates; (3) leadership – three lines, the headline, the risk and the ask. Keep every version truthful to the same facts; only change tone and detail.

4. Check the tone yourself. Read the leadership version as if you were the executive: is it clear, honest and free of jargon and spin? Adjust it — you own how the team communicates, not the AI.

**5. Generate test scenarios from acceptance criteria.** Feed your Lab 4 acceptance criteria in with the prompt below.

For the user story and acceptance criteria below:

<PASTE A KEY STORY AND ITS GIVEN/WHEN/THEN CRITERIA>

Task: write test scenarios that would verify each acceptance criterion, including edge cases and error conditions (offline, permissions denied, an empty state, a shared-habit conflict). Format: a numbered checklist of 'Given/When/Then' test cases. Note any acceptance criterion that is not actually testable as written.

**6. Run a definition-of-done check.** Ask the AI to test a story against your Definition of Done, then apply your own judgement.

Here is our Definition of Done:

<PASTE OR LIST YOUR DEFINITION OF DONE>

and here is a story we think is finished:

<PASTE THE STORY AND WHAT WAS DELIVERED>

Task: check it against each Definition-of-Done item and list what is met, not met or unclear. Do not pass anything you cannot verify from the information given.

**7. Save the documentation, communication and testing section of your Tempo-2.0-Playbook folder:** the reviewed release notes and templates, the three audience-tailored updates, and the test scenarios and definition-of-done check.

### Test it

You have AI-assisted sprint documentation with invented detail removed and gaps marked '[to confirm]', one update correctly re-pitched for the team, stakeholders and leadership, and test scenarios plus a definition-of-done check generated from your acceptance criteria and reviewed by you — all saved to your playbook.

**Note:** Full commands and screenshots are in labs/lab-09-\*.md. Use only projects, data and notes you are authorised to use. Do not paste confidential customer data, personal information, credentials or proprietary code into a public AI assistant. Use the supplied Tempo 2.0 project brief rather than real client material, treat every AI output as a first draft to be reviewed and fact-checked, and be transparent with your team about what was AI-assisted before it goes into a real backlog, plan, report or decision.

## Topic 04 — AI for Reporting, Retrospectives and Improvement (25%)

Generating status reports and dashboards · Analysing velocity and metrics with AI · Running AI-assisted retrospectives · Driving continuous improvement

## Key concepts

- Status reports with AI — the AI turns sprint data and notes into a clear status report — progress against goal, completed and outstanding work, risks and next steps — pitched at the right level for its reader.
- Dashboards and summaries — the AI helps design and describe a simple delivery dashboard (goal, scope, burndown, velocity, risks) and writes the narrative that explains what the numbers mean.
- Analysing velocity — given past sprint data, the AI analyses velocity trends, flags a rising or falling trend, and helps forecast how much scope is realistic for coming sprints, with its assumptions stated.
- Reading the metrics honestly — the AI helps interpret metrics (velocity, throughput, cycle time, completion rate) as signals for conversation and improvement, not as targets to game or sticks to beat the team with.
- AI-assisted retrospectives — the AI helps design a retrospective, group and theme the team's raw input into patterns, and surface candid discussion points, so the facilitator can focus on the conversation.
- From retro to actions — the AI drafts a small set of specific, owned, time-bound improvement actions from the retrospective themes, which the team reviews, commits to and carries into the next sprint.
- Continuous improvement — the AI helps track improvement actions across sprints, spot recurring themes, and keep an honest record of what the team tried and what changed, feeding the inspect-and-adapt loop.
- Psychological safety — retrospective input is sensitive; you anonymise and aggregate it, keep it inside the team, and use AI to find themes, never to judge or expose individuals.
- Closing the loop — reporting, metrics and retrospectives only add value when they change what the team does next; the whole point of AI here is faster insight so the team spends its time acting on it.

### Lab 10 — Generate Status Reports and a Delivery Dashboard

Learning outcome: Use AI to turn sprint data and notes into a clear status report pitched for its reader, and to design and describe a simple delivery dashboard with the narrative that explains what the numbers mean.

Goal: At the end of a sprint the team must tell its story to others. In this lab you use an assistant to produce reporting that is clear and honest. You feed the sprint's goal, completed and outstanding work, risks and next steps to the AI and have it draft a status report — then re-pitch it for the right reader, from a one-page stakeholder update to a three-line leadership summary. You then design a simple delivery dashboard: you ask the AI which few metrics matter (goal, scope, burndown, velocity, risks), how to lay them out, and — most importantly — to write the narrative that explains what the numbers actually mean, so a dashboard informs rather than misleads. You review every figure the AI states against your real data. **BUILDING BLOCK** — what you create in this lab

becomes a section of your Tempo 2.0 Agile AI Playbook, the connected project you assemble across all 12 labs.

## What you'll build

A reviewed Tempo 2.0 sprint status report pitched for its audience, and a simple delivery-dashboard design (the few metrics that matter, a layout, and the plain-language narrative that explains them) — saved as the reporting section of your playbook. (Tools: Any assistant, the status-report template from your prompt library, audience-pitched reporting, dashboard design, a metrics narrative.)

## Step-by-step

1. Gather your sprint data: the sprint goal, what was completed and what slipped, your risk log from Lab 8, and the burndown from Lab 7. Grab your status-report template from your prompt library.
2. Draft the status report. Paste the prompt below with your data.

You are a delivery lead writing a sprint status report for stakeholders. Here is our Tempo 2.0 Sprint 1 data:

<PASTE GOAL, COMPLETED, OUTSTANDING, RISKS, NEXT STEPS>

Task: write a one-page status report with these sections – Sprint goal and whether we met it, What we delivered, What is outstanding and why, Risks and mitigations, Next sprint focus. Format: clear headings and short bullets. Constraints: be honest about what slipped, use only the data given, and do not overstate progress.

3. Re-pitch for leadership. Ask the AI to compress the same report for executives with the prompt below.

Compress that status report into a three-line leadership summary: line 1 the headline (did we meet the sprint goal), line 2 the main risk, line 3 the ask or decision needed. Keep it truthful to the full report.

4. Fact-check the report. Check every number and claim against your real data — completed points, dates, risk status. Correct anything the AI rounded, guessed or overstated. A status report the team cannot stand behind is worse than none.
5. Design a delivery dashboard. Ask the AI what to show and how, using the prompt below.

Design a simple one-screen delivery dashboard for the Tempo 2.0 release that a mixed audience could read at a glance. Task: recommend the few metrics that matter most (for example sprint goal status, scope completed vs planned, burndown, velocity trend, top risks), describe a clean layout for them, and say what each metric should and should not be used to conclude. Do not add vanity metrics.

6. Write the metrics narrative. Have the AI explain the numbers in words, then review it.

Write a short narrative to sit beside the dashboard that explains what this sprint's numbers actually mean for a non-technical reader – what is going well, what to watch, and what the numbers do NOT tell us. Base it on:

<PASTE YOUR REAL SPRINT METRICS>

Be honest and avoid making the metrics sound better than they are.

**7. Review and save.** Confirm the dashboard would inform, not mislead, and that the narrative is honest. Save the status report, dashboard design and narrative as the reporting section of your Tempo-2.0-Playbook folder.

### Test it

You have a reviewed Tempo 2.0 status report with an audience-pitched leadership summary and every figure fact-checked against real data, plus a simple delivery-dashboard design with an honest narrative explaining what the metrics do and do not mean — all saved to your playbook.

**Note:** Full commands and screenshots are in labs/lab-10-\*.md. Use only projects, data and notes you are authorised to use. Do not paste confidential customer data, personal information, credentials or proprietary code into a public AI assistant. Use the supplied Tempo 2.0 project brief rather than real client material, treat every AI output as a first draft to be reviewed and fact-checked, and be transparent with your team about what was AI-assisted before it goes into a real backlog, plan, report or decision.

## Lab 11 — Analyse Velocity and Metrics with AI

**Learning outcome:** Use AI to analyse past sprint data — velocity trend, throughput, cycle time and completion rate — to forecast realistic scope, and to interpret the metrics honestly as signals for conversation, not targets to game.

**Goal:** Metrics only help if they are read well. In this lab you use an assistant as an analyst on the supplied Tempo history — several sprints of velocity and delivery data. You ask the AI to analyse the velocity trend, flag whether it is rising, falling or noisy, and forecast how much scope is realistic for the next sprints, with its assumptions stated. You have it interpret throughput, cycle time and completion rate as signals — what each suggests to talk about — and you explicitly guard against misusing them: metrics are for the team to improve, not targets to hit or sticks to beat people with. You review every calculation and reading, because a confident but wrong analysis is worse than none. **BUILDING BLOCK** — what you create in this lab becomes a section of your Tempo 2.0 Agile AI Playbook, the connected project you assemble across all 12 labs.

### What you'll build

An AI-assisted analysis of the Tempo velocity and delivery metrics — a velocity-trend read, a realistic scope forecast with assumptions, and an honest interpretation of throughput, cycle time and completion rate as signals — reviewed by you and saved.

(Tools: Any assistant, the sprint metrics history, velocity-trend analysis, scope forecasting, honest metric interpretation.)

### Step-by-step

1. Open the supplied sprint metrics history for Tempo (labs/reference-pack/) — several sprints of committed vs completed points, and any throughput or cycle-time figures. Paste it into any assistant.
2. Analyse the velocity trend. Use the prompt below.

You are an Agile delivery analyst. Here is our Tempo sprint history:

<PASTE THE SPRINT METRICS>

Task: analyse our velocity trend across these sprints – is it rising, falling, stable or too noisy to tell? Show the average and the range, and explain what could be driving the pattern. Constraints: state clearly if the data is too little to be reliable, and do not present a trend more confident than the data supports.

3. Forecast realistic scope. Ask the AI to project the next sprints with assumptions stated.

Based on that velocity analysis, forecast how many story points we could realistically commit to in each of the next 2 sprints, giving a conservative and an optimistic figure. State every assumption (team availability, no major disruption, similar work type). Make clear this is a planning aid, not a promise.

4. Verify the maths. Recompute the average velocity yourself from the raw numbers and compare it to the AI's. If they differ, find out why. Never report a metric you have not checked.
5. Interpret the wider metrics as signals. Use the prompt below for throughput, cycle time and completion rate.

Interpret our throughput, cycle time and sprint completion rate as signals for a retrospective conversation, not as performance targets. For each metric: what a healthy pattern looks like, what our numbers might be signalling, and one question the team should discuss. Explicitly warn against any way this metric could be gamed or misused to pressure individuals.

6. Add the human read. Write two or three lines of your own on what the metrics really suggest for Tempo, and note one thing the numbers do not capture (for example quality, morale, or hidden rework). Metrics inform the conversation; they do not end it.
7. Save the metrics analysis as a section of your Tempo-2.0-Playbook folder: the velocity-trend read, the scope forecast with assumptions, the signals interpretation, and your own honest commentary — ready to feed the retrospective.

### Test it

You have an AI-assisted metrics analysis for Tempo — a velocity-trend read you verified by re-computing the average, a scope forecast with explicit assumptions, and an honest

interpretation of throughput, cycle time and completion rate as signals (not targets) — with your own commentary, saved to your playbook.

**Note:** Full commands and screenshots are in labs/lab-11-\*.md. Use only projects, data and notes you are authorised to use. Do not paste confidential customer data, personal information, credentials or proprietary code into a public AI assistant. Use the supplied Tempo 2.0 project brief rather than real client material, treat every AI output as a first draft to be reviewed and fact-checked, and be transparent with your team about what was AI-assisted before it goes into a real backlog, plan, report or decision.

## Lab 12 — Run an AI-Assisted Retrospective and Plan Continuous Improvement

Learning outcome: Use AI to design a retrospective, theme the team's anonymised input into patterns, surface candid discussion points, and turn the themes into a small set of specific, owned, time-bound improvement actions — completing the end-to-end playbook.

Goal: A sprint ends by learning from it. In this final lab you use an assistant to make a retrospective sharper and its follow-through real, while protecting the people in it. You ask the AI to design a retrospective format suited to this sprint, then feed it the team's raw, anonymised retro input and have it group and theme the feedback into patterns and surface honest discussion points — the facilitator's prep done in seconds. Crucially, you handle the input responsibly: it is anonymised and aggregated, kept inside the team, and used to find themes, never to judge individuals. From the themes the AI drafts a few specific, owned, time-bound improvement actions, which you review and commit to, and you set up a simple way to track them across sprints so the inspect-and-adapt loop actually closes. This assembles your complete Tempo 2.0 Agile AI Playbook. BUILDING BLOCK — what you create in this lab becomes a section of your Tempo 2.0 Agile AI Playbook, the connected project you assemble across all 12 labs.

### What you'll build

An AI-assisted retrospective — a fit-for-purpose format, the team's anonymised input themed into patterns with candid discussion points, and a small set of specific, owned, time-bound improvement actions with a way to track them — completing your end-to-end Tempo 2.0 Agile AI Playbook. (Tools: Any assistant, retrospective design, anonymised theming of team input, discussion-point surfacing, improvement-action planning, continuous-improvement tracking.)

### Step-by-step

1. Design the retrospective. Ask the AI for a format suited to this sprint, using the prompt below.

You are an Agile facilitator. Our Tempo 2.0 Sprint 1 had a strong start but slipped on the reminder-engine work and had a dependency block. Task: suggest a retrospective format suited to this sprint (for example Start/Stop/Continue,

Glad/Sad/Mad, or 4Is), with a short agenda and 4-5 good opening questions. Explain in one line why this format fits this sprint.

**2. Prepare the input responsibly.** Take the supplied raw retro input (labs/reference-pack/) and confirm it is anonymised before you use it — no names, no blame. If your own team's input had names, strip them first. Note this as the rule: retro input is aggregated and stays inside the team.

**3. Theme the feedback.** Feed the anonymised input in with the prompt below.

Here is the anonymised raw input from our sprint retrospective:

<PASTE THE ANONYMISED RETRO INPUT>

Task: group it into 4-6 themes, showing how many comments support each and summarising the point of view within each theme fairly. Format: Theme | Summary | How many mentioned it. Constraints: keep it anonymous and neutral, represent minority views too, and do not single out or infer any individual.

**4. Surface candid discussion points.** Ask the AI to turn themes into questions the team should actually discuss.

From those themes, suggest 4-5 candid but constructive discussion points for the retrospective — the things the team most needs to talk about honestly to improve, phrased as questions, not accusations. Focus on the system and process, not people.

**5. Draft improvement actions.** Convert the discussion into commitments with the prompt below.

Turn the top themes into a SMALL set (3-4) of improvement actions for next sprint. Each must be specific, have a suggested owner role, and be time-bound (done by when). Format: Action | Owner (role) | By when | The theme it addresses. Constraints: keep it to a few actions the team can actually do — a long list improves nothing.

**6. Commit and set up tracking as a human.** Review the actions with the team's hat on: are they genuinely doable, do they address the real themes? Adjust, assign real owners, and set up a simple tracker (a short list carried into next sprint's retro) so you can check what actually changed — closing the inspect-and-adapt loop.

**7. Assemble the complete playbook.** Save the retrospective format, themed feedback, discussion points and improvement-action tracker as the final section of your Tempo-2.0-Playbook folder. Then review the whole folder end to end — toolkit, prompts, responsible-AI checklist, stories, backlog, plan, tracking, risks, docs, reports, metrics and retro — as one connected Tempo 2.0 Agile AI Playbook. This is the deliverable the course set out to build.

## Test it

You have an AI-assisted retrospective — a fit-for-purpose format, the team's anonymised input themed into patterns with candid discussion points, and 3-4 specific, owned, time-bound improvement actions with a tracker — handled responsibly

(anonymised, no blame), and your complete end-to-end Tempo 2.0 Agile AI Playbook is assembled and reviewed.

**Note:** Full commands and screenshots are in labs/lab-12-\*.md. Use only projects, data and notes you are authorised to use. Do not paste confidential customer data, personal information, credentials or proprietary code into a public AI assistant. Use the supplied Tempo 2.0 project brief rather than real client material, treat every AI output as a first draft to be reviewed and fact-checked, and be transparent with your team about what was AI-assisted before it goes into a real backlog, plan, report or decision.

## Wrap-Up

You have taken one product — Tempo 2.0 — through an entire AI-assisted agile delivery across two days, from setting up an AI toolkit to running a retrospective, using ChatGPT, Claude and Gemini as a drafting and analysis partner while keeping every decision human.

### What you built

- An AI toolkit for agile work — ChatGPT, Claude and Gemini set up, a reusable agile prompt library, and a responsible, human-in-the-loop checklist.
- A planned delivery — INVEST-quality user stories and acceptance criteria, a groomed and MoSCoW-prioritised product backlog, and a sprint plan, estimates and release roadmap.
- A running sprint — stand-up summaries and progress tracking, a risk, blocker and dependency log, and AI-assisted documentation, communication and test scenarios.
- A closed-out sprint — stakeholder status reports and a delivery dashboard, a velocity and metrics analysis, and an AI-assisted retrospective with a concrete continuous-improvement plan.
- One connected Tempo 2.0 Agile AI Playbook that carries the whole delivery, section by section.

### What to do next

- Rebuild the playbook for a real, non-confidential project of your own using the same prompts and templates.
- Introduce your saved prompt library to your team so everyone drafts stories, summaries and reports the same way.
- Keep your responsible-AI checklist visible in every sprint — review, fact-check and protect data every time.
- Always keep the team's judgement in the loop: the AI drafts and analyses, but the Product Owner, Scrum Master and team own every decision.

## Next Steps

- First pass: complete every lab yourself, following the steps and verifying each 'Test it' check.
- Second pass: rerun the key labs on your own real, non-confidential project, from setup and prompts to a retrospective.
- Introduce your prompt library and responsible-AI checklist to your team so the practice sticks beyond the course.
- Review each lab's detailed steps in this guide and re-create the playbook in your own AI assistant.

## Glossary

- **Agile** — An iterative approach to delivering value in small increments, inspecting and adapting frequently rather than following one big up-front plan.
- **Scrum** — The most common Agile framework, with fixed roles, events and artifacts organised around short iterations called sprints.
- **Sprint** — A short, fixed-length iteration (commonly 1–4 weeks) in which the team delivers a usable increment toward the product goal.
- **Product Owner** — The Scrum role accountable for the product backlog, its priority, and maximising the value the team delivers.
- **Scrum Master** — The Scrum role that facilitates the process, removes impediments, and helps the team improve; the main AI user in several of these labs.
- **Product backlog** — The ordered, evolving list of everything that might be built — features, fixes and work — from which sprints are planned.
- **Sprint backlog** — The set of backlog items the team commits to for a sprint, plus the plan (tasks) for delivering them.
- **User story** — A short, user-centred description of a feature in the form 'As a <role>, I want <goal>, so that <benefit>'.
- **INVEST** — A checklist for good user stories — Independent, Negotiable, Valuable, Estimable, Small, Testable.
- **Acceptance criteria** — The conditions, often written Given/When/Then, that a story must meet to be considered done.
- **Backlog grooming (refinement)** — The ongoing work of clarifying, splitting, merging, estimating and ordering backlog items so they are ready for planning.
- **MoSCoW** — A prioritisation method sorting work into Must have, Should have, Could have and Won't have (this time).
- **Story point** — A unit of relative estimate for the effort of a story, used instead of hours to size work by comparison.
- **Velocity** — The amount of work (usually story points) a team completes per sprint, used as a trend to forecast realistic scope.
- **Burndown** — A chart or narrative showing how much work remains in a sprint or release over time.

- **Daily stand-up** — The short daily Scrum event where the team syncs on progress, plans and blockers.
- **Blocker / impediment** — Anything stopping a team member or story from progressing, which the Scrum Master works to remove.
- **Dependency** — A relationship where one item, team or external party must be done or available before another can proceed.
- **Definition of Done** — The team's shared, agreed checklist of what must be true for a story or increment to count as complete.
- **Retrospective** — The Scrum event at the end of a sprint where the team reflects on how it worked and agrees improvements.
- **Continuous improvement** — The ongoing inspect-and-adapt practice of making small, owned changes each sprint based on evidence and reflection.
- **Roadmap** — A higher-level, multi-sprint view of the themes and releases planned over time, kept deliberately flexible.
- **Release plan** — A plan grouping backlog items into releases with target dates, dependencies and milestones.
- **Generative AI assistant** — A general-purpose chat tool (ChatGPT, Claude, Gemini) that generates text drafts and analysis from a prompt.
- **Prompt** — The instruction you give the assistant; a structured prompt with role, context, task, format and constraints produces a far better draft than a vague one.
- **Prompt library** — A saved, reusable set of prompt templates for the recurring agile tasks, shared across the team.
- **Human-in-the-loop** — Keeping a person responsible for reviewing, correcting and approving every AI output before it is used.
- **Hallucination** — A confident but false or invented statement from an AI, which is why every output must be fact-checked.
- **Responsible AI use** — Using AI safely and ethically — protecting sensitive data, checking facts, being fair, and being transparent about AI assistance.